pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

APIs, Keys, and Secrets

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: a key in your name

The Desk Lead's Note

From today the morning numbers come from FRED, pulled by code. FRED wants a key on every request, in your name. Before any code touches a real key, show me what FRED sends back, checked against the Treasury file we trust. Get your own key and give it a home outside every file, notebook, and commit. Write the one function that finds it, tested. Pull the 10-year live and tie it to the snapshot. Then show me a key leaking, and the fix.

Course 2 ended with a bond math module of your own, tested against Excel. Course 3 brings the market's own numbers in from outside, the same way: one tested function a day, in a new module.

FRED is the Federal Reserve Bank of St. Louis's free data service. Every cell runs without a key, the saved outputs are that run, and every demo uses a fake key.

Start the Day

```
work = start_day(1)
Work folder pfi_course3_01_k4w7q2xr: 3 data files, Course 2's bondmath.py, no marketdata.py yet, a Git repository with no commits
```

Every request carries a key in your name: today it gets a home outside the code before any code needs it.

1.1 What FRED sends back

An Answer, Read as Text

```
with open('course3_fred_format_dgs10.json', encoding='utf-8') as file:
    sample_lines = file.readlines()
print(''.join(sample_lines[:19]) + '...\n' + ''.join(sample_lines[-6:]))
```


```
{
  "realtime_start": "2026-10-02",
  "realtime_end": "2026-10-02",
  "observation_start": "2026-08-03",
  "observation_end": "2026-09-30",
  "units": "lin",
  "output_type": 1,
  "file_type": "json",
  "order_by": "observation_date",
  "sort_order": "asc",
  "count": 43,
  "offset": 0,
  "limit": 100000,
  "observations": [
   {
   "realtime_start": "2026-10-02",
   "realtime_end": "2026-10-02",
   "date": "2026-08-03",
   "value": "4.70"
...
   "realtime_end": "2026-10-02",
   "date": "2026-09-30",
   "value": "5.29"
   }
  ]
}
```


The course built this file from Treasury's values in FRED's exact layout: 43 dated observations, from 4.70 on 3 August 2026 to 5.29 on 30 September.

The Same Day in the Snapshot

```
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
print(f'{len(treasury):,} market days, {treasury.index[0].date()} to {treasury.index[-1].date()}')
print('10 Yr on 2026-09-30:', treasury.loc['2026-09-30', '10 Yr'], 'percent')
```

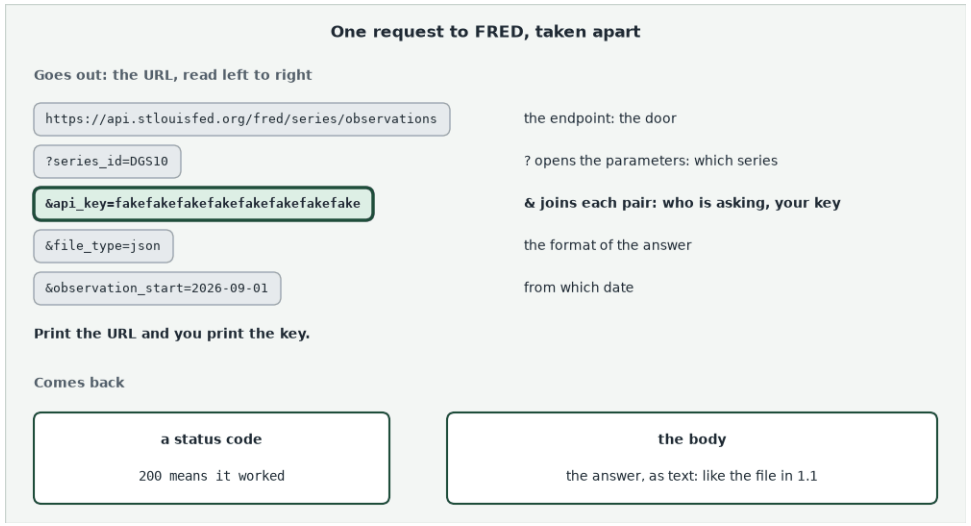
```
2,940 market days, 2015-01-02 to 2026-10-02
10 Yr on 2026-09-30: 5.29 percent
```

This is the course's snapshot: the U.S. Treasury's par yield curve, public domain, frozen on 2 October 2026. Every analysis uses it, so saved outputs never change.



FRED's answer is text you can read, and its 5.29 on 30 September is the Treasury file's 5.29.

1.2 The request, taken apart




```
FAKE_KEY = 'fakefakefakefakefakefakefakefake'
FRED_URL = 'https://api.stlouisfed.org/fred/series/observations'
print(len(FAKE_KEY), 'characters')
32 characters
```

FRED documents a key as "a 32 character lower-cased alpha-numeric string" (https://fred.stlouisfed.org/docs/api/api_key.html, read 2026-10-04).

The fake key has the shape and opens nothing; it goes under "DEMO_API_KEY", never "FRED_API_KEY".

Build the URL

```
params = {'series_id': 'DGS10', 'api_key': FAKE_KEY, 'file_type': 'json',
          'observation_start': '2026-09-01'}
request_url = FRED_URL + '?' + '&'.join([f'{name}={value}' for name, value in params.items()])
print(request_url)
https://api.stlouisfed.org/fred/series/observations?series_id=DGS10&api_key=fakefakefakefakefakefakefakefake&file_type=json&observation_start=2026-09-01
```

The key rides inside the URL: print, log, or quote a URL and the key goes with it.

Excel side by side: the same URL

Excel side by side. A1 holds "DGS10", B1 the fake key, C1 the text "2015-01-01".

Job	Python	Result
The URL from its parts	`request_url, starting 2015-01-01`	`the full URL, fake key inside`
Encode text for a URL	`quote('a b&c=d/e?f', safe='')`	`a%20b%26c%3Dd%2Fe%3Ff`

```
The URL from its parts
="https://api.stlouisfed.org/fred/series/observations?series_id="&A1&"&api_key="&B1&"&file_type=json&observation_start="&ENCODEURL(C1)
Encode text for a URL
=ENCODEURL("a b&c=d/e?f")
```

Excel's "&" operator joins text as Python's "+" does; the "&" inside the quotes is the URL's own separator. "ENCODEURL" leaves an ISO date unchanged.

Same URL as Excel's

```
from urllib.parse import quote
url_2015 = request_url.replace('2026-09-01', '2015-01-01')
excel_url = (FRED_URL + '?series_id=DGS10&api_key=fakefakefakefakefakefakefakefake'
             + '&file_type=json&observation_start=2015-01-01')
print('Same URL as Excel:', url_2015 == excel_url)
print(quote('a b&c=d/e?f', safe=''))
Same URL as Excel: True
a%20b%26c%3Dd%2Fe%3Ff
```

The key rides inside the URL, so anything that prints a URL prints your key; FRED's error messages never repeat it.

1.3 Your own key, kept like a password

The key is free:

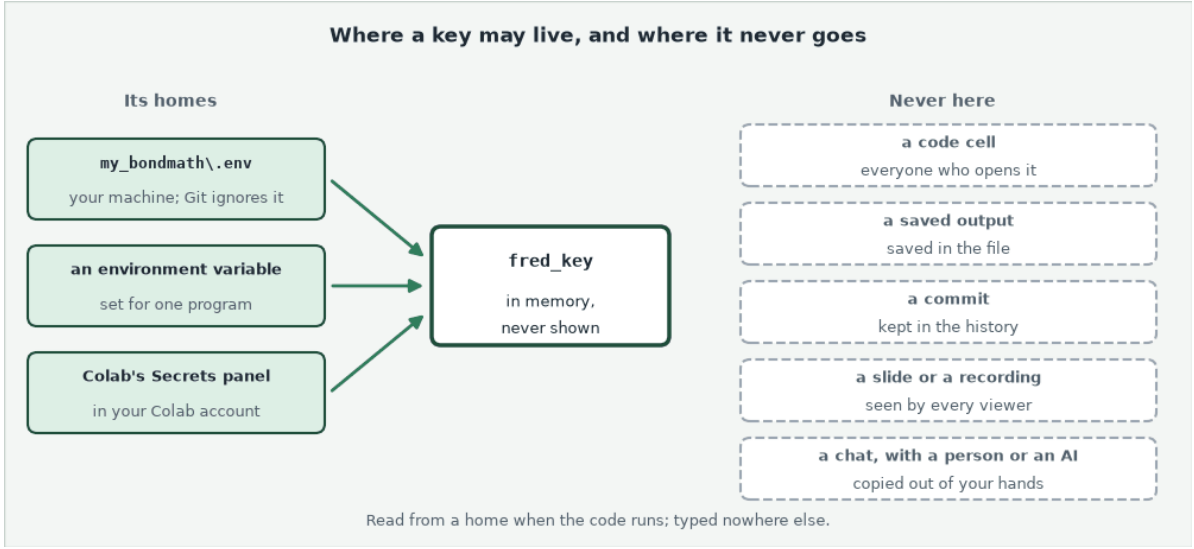
1. Sign in, or create a free account, at https://fredaccount.stlouisfed.org/apikeys: FRED shows and issues keys only to a signed-in account.

2. Request a key on that page.

3. Copy it, and paste it nowhere yet: Bring It Home gives it its place.

One Key per Person

FRED's key page (https://fred.stlouisfed.org/docs/api/api_key.html, read 2026-10-04) says so, and sets one key per person: "All users of an application shall use their own API key", and "Developers should request a distinct API key for each application they build."



1.4 Environment variables

Every program runs with named text values around it, its environment variables.

Is DEMO_API_KEY There?

```
print('DEMO_API_KEY' in os.environ)
print(os.environ.get('DEMO_API_KEY'))
False
None
```

Set It, Compare, Remove It

```
os.environ['DEMO_API_KEY'] = FAKE_KEY
print('The value is the fake key:', os.environ['DEMO_API_KEY'] == FAKE_KEY)
del os.environ['DEMO_API_KEY']
print('DEMO_API_KEY' in os.environ)
The value is the fake key: True
False
```

Set this way, it lives only in this notebook's Python. PowerShell sets one in two ways, shown here as text:

```
# this terminal window only
$env:DEMO_API_KEY = "..."
# saved for your Windows user
[Environment]::SetEnvironmentVariable("DEMO_API_KEY", "...", "User")
```

Both land in PowerShell's history file, the list the up arrow walks through, so a real key goes in a file typed in an editor.

A secret in the environment sits outside the code: the code asks for it by name.

1.5 The .env file and .env.example

A ".env" file holds "NAME=value" lines, one secret to a line: a widely used convention. Its values never go into Git; ".env.example", the same names with empty values, does.

```
%%writefile .env
# today's drill: the course's fake key, never a real one
DEMO_API_KEY=fakefakefakefakefakefakefakefake
Writing .env
```

```
%%writefile .env.example
# the names of the secrets this folder needs, with no values. Copy it to .env and fill in the values.
DEMO_API_KEY=
Writing .env.example
```

The plainest form every tool reads: no spaces around "=", no quotes; a "#" line is a comment.

Read It

```
from dotenv import dotenv_values
values = dotenv_values('.env')
print('Names:', list(values))
print('The value is the fake key:', values['DEMO_API_KEY'] == FAKE_KEY)
print('DEMO_API_KEY in os.environ:', 'DEMO_API_KEY' in os.environ)
Names: ['DEMO_API_KEY']
The value is the fake key: True
DEMO_API_KEY in os.environ: False
```

The environment is untouched. "load_dotenv" would copy the values into "os.environ"; the course reads the named file and copies nothing.

".env" holds the value, ".env.example" holds the names, and only the names will be committed.

1.6 .gitignore before the first commit

Course 2's notebook 06 used ".gitignore" for Python's folders; here it keeps ".env" out, written before the first commit: 1.10 shows why. "start_day(1)" made the work folder an empty repository.

```
%%writefile .gitignore
# secrets: your .env holds real keys and is never committed
.env
# downloaded data and databases: rebuilt, never part of the record
cache/
*.db
# folders Python and pytest make
__pycache__/
.pytest_cache/
Writing .gitignore
```

"cache/" and "*.db" wait for notebook 05's cache and notebook 11's database.

What Git Sees

```
!git -C "{work}" status --short --ignored
?? .env.example
?? .gitignore
!! .env
!! bondmath.py
!! course3_excel_reference.csv
!! course3_fred_format_dgs10.json
!! treasury_par_yields.csv
```

"!!" marks an ignored name, as in Course 2's notebook 11: ".env" by ".gitignore", the copied data files and "bondmath.py" by the work folder's own ignore list. "??" marks the files that belong in the record.

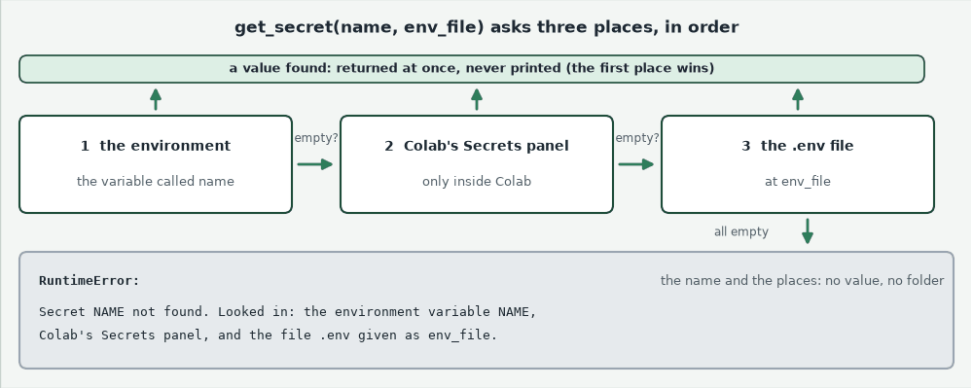
Which Line Keeps .env Out?

```
!git -C "{work}" check-ignore -v .env
.gitignore:2:.env    .env
```

".env" is ignored before anything is committed, and "check-ignore" names the line that does it.

1.7 get_secret, docstring first

```
%%writefile -a marketdata.py
def get_secret(name: str, env_file: str | Path) -> str:
    """Return a secret such as an API key, looked up in three places in a fixed order.
    Units: none; the secret is text. Convention: the first place that holds a non-empty value wins:
    1. the environment variable `name`;
    2. Colab's Secrets panel (google.colab.userdata), only when running in Colab;
    3. the .env file at `env_file`, read with dotenv.dotenv_values, which does not change os.environ.
    The value is returned and never printed. Raises RuntimeError naming the secret and the three places
    looked, never a value and never the full path of the file.
    """
    value = os.environ.get(name)
    if value:
        return value
    try:
        from google.colab import userdata
    except ImportError:
        userdata = None
    if userdata is not None:
        try:
            value = userdata.get(name)
        except (userdata.SecretNotFoundError, userdata.NotebookAccessError):
            value = None
        if value:
            return value
    env_path = Path(env_file)
    if env_path.is_file():
        from dotenv import dotenv_values
        value = dotenv_values(env_path).get(name)
        if value:
            return value
    raise RuntimeError(
        f"Secret {name} not found. Looked in: the environment variable {name}, "
        f"Colab's Secrets panel, and the file {env_path.name} given as env_file."
    )
```



Read the docstring before you run the cell. An import inside a function runs only when that line is reached, so Colab never needs "python-dotenv".

Find the Drill's Key

```
import marketdata as md
secret = md.get_secret('DEMO_API_KEY', env_file='.env')
print(f'Found the fake key: {secret == FAKE_KEY}, {len(secret)} characters')
Found the fake key: True, 32 characters
```

No variable and no Colab, so the file answered.

A secret that is nowhere, four tests, a commit

```
try:
    md.get_secret('NO_SUCH_SECRET', env_file='.env')
except RuntimeError as error:
    print(error)
Secret NO_SUCH_SECRET not found. Looked in: the environment variable NO_SUCH_SECRET, Colab's Secrets panel, and the file .env given as env_file.
```

A Secret That Is Nowhere

The name and the three places: no value, no folder.

```
import pytest
import marketdata as md
FAKE_KEY = "fakefakefakefakefakefakefakefake"
DEMO_NAME = "DEMO_API_KEY"
def test_get_secret_reads_environment(monkeypatch, tmp_path):
    monkeypatch.setenv(DEMO_NAME, FAKE_KEY)
    assert md.get_secret(DEMO_NAME, env_file=tmp_path / ".env") == FAKE_KEY
def test_get_secret_reads_env_file(monkeypatch, tmp_path):
    monkeypatch.delenv(DEMO_NAME, raising=False)
    env_file = tmp_path / ".env"
    env_file.write_text(f"{DEMO_NAME}={FAKE_KEY}\n", encoding="utf-8")
    assert md.get_secret(DEMO_NAME, env_file=env_file) == FAKE_KEY
def test_get_secret_environment_wins(monkeypatch, tmp_path):
    env_file = tmp_path / ".env"
    env_file.write_text(f"{DEMO_NAME}=value-from-the-file\n", encoding="utf-8")
    monkeypatch.setenv(DEMO_NAME, FAKE_KEY)
    assert md.get_secret(DEMO_NAME, env_file=env_file) == FAKE_KEY
def test_get_secret_missing_raises_without_value(monkeypatch, tmp_path):
    monkeypatch.delenv(DEMO_NAME, raising=False)
    env_file = tmp_path / ".env"
    env_file.write_text(f"OTHER_API_KEY={FAKE_KEY}\n", encoding="utf-8")
    with pytest.raises(RuntimeError) as caught:
        md.get_secret(DEMO_NAME, env_file=env_file)
    message = str(caught.value)
    assert DEMO_NAME in message and "environment" in message and ".env" in message
    assert FAKE_KEY not in message
    assert str(tmp_path) not in message
```

A pytest fixture is a tool a test receives by naming it as an argument. "monkeypatch" sets or removes an environment variable for one test only; "tmp_path" gives each test its own empty folder; "write_text" writes a whole file.

```
....   [100%]
4 passed in 0.50s
```

```
!git -C "{work}" add .gitignore .env.example marketdata.py test_marketdata.py
subject = 'Notebook 01: get_secret, a .gitignore, and .env.example'
body = 'Course 3 pulls data with a key, so the key gets a home outside the code first.'
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
!git -C "{work}" show --stat --oneline HEAD
5ca8845 Notebook 01: get_secret, a .gitignore, and .env.example
 .env.example   |  2 ++
 .gitignore     |  8 ++++++++
 marketdata.py  | 58 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py | 50 ++++++++++++++++++++++++++++++++++++++++++++++++++
 4 files changed, 118 insertions(+)
```

"get_secret" finds a secret in three places and never shows it, "4 passed" proves it, and the first commit holds four files and no ".env".

1.8 Colab's Secrets panel, 1.9 the live pull

1.8 Colab's Secrets Panel

Colab keeps secrets in a panel, not a ".env":

1. Open Secrets, the key icon on the left.
2. Add "FRED_API_KEY" with your key as its value.
3. Turn on Notebook access.

The secret stays in your Colab account, so sharing the notebook does not share the key. Never upload a ".env" to Colab.

In Colab the key lives in your account's Secrets panel, and "get_secret" asks there second.

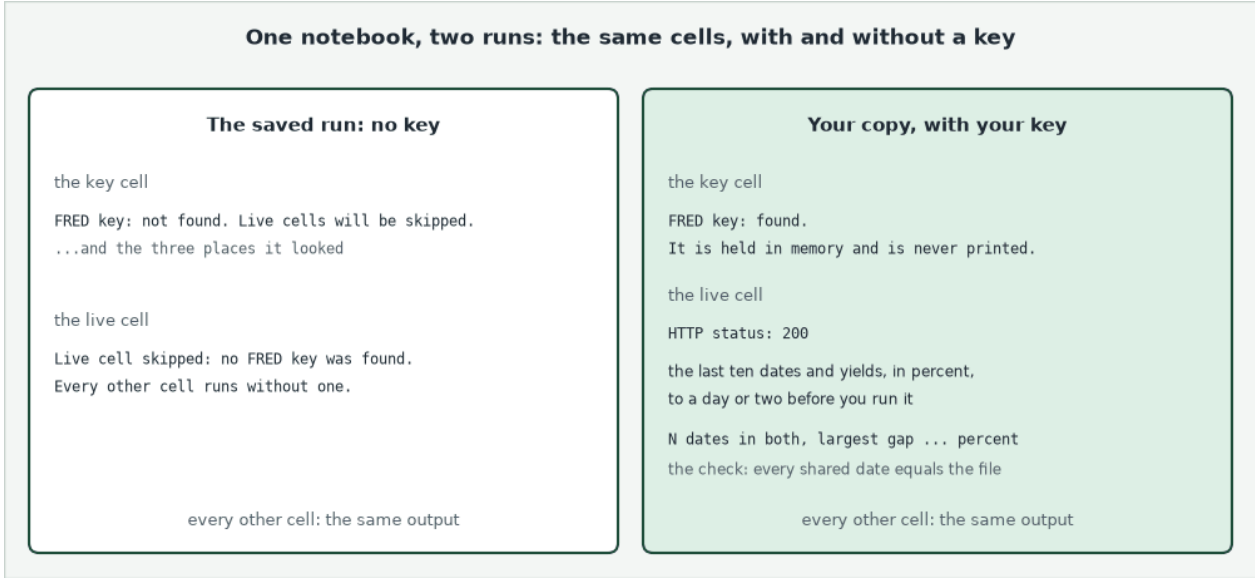
The Key Cell

```
try:
    fred_key = md.get_secret("FRED_API_KEY", env_file=ENV_FILE)
    print("FRED key: found. It is held in memory and is never printed.")
except RuntimeError as error:
    fred_key = None
    print(f"FRED key: not found. Live cells will be skipped. {error}")
```

A key cell puts get_secret's answer for "FRED_API_KEY" in "fred_key", or "None"; each live cell checks for "None" first; both are tagged "live-fred".

Never type "fred_key" alone in a cell, or "print" it: Jupyter shows a cell's last value and saves it in the file.

Never print a URL or a response's ".url": the key is inside it.



This product uses the FRED® API but is not endorsed or certified by the Federal Reserve Bank of St. Louis.

1.10 When a key leaks

A key leaks when it lands where others can read it: a commit, a pushed repository, a saved output, a screenshot, a recording, a chat.

The Mistake, on Purpose

```
%%writefile settings.py
# a settings file with a key typed into it: the mistake the drill makes on purpose
DEMO_API_KEY = "fakefakefakefakefakefakefakefake"
Writing settings.py
```

```
!git -C "{work}" add settings.py
!git -C "{work}" commit -q -m "Add settings"
!git -C "{work}" log --oneline
db1938d Add settings
5ca8845 Notebook 01: get_secret, a .gitignore, and .env.example
```

Ignore It, Then Change It

Someone notices, lists the name in ".gitignore", and edits the file.

```
%%writefile -a .gitignore
settings.py
%%writefile -a settings.py
# one more line
```

Delete It the Git Way

```
!git -C "{work}" rm -q settings.py
!git -C "{work}" add .gitignore
!git -C "{work}" commit -q -m "Remove settings.py and ignore it"
!git -C "{work}" ls-files
.env.example
.gitignore
marketdata.py
test_marketdata.py
```

Search the History

```
!git -C "{work}" log --oneline -S {FAKE_KEY} -- settings.py
11224b4 Remove settings.py and ignore it
db1938d Add settings
```

Two commits: the one that added the key and the one that removed it.

Read it back, then rotate

Read It Back

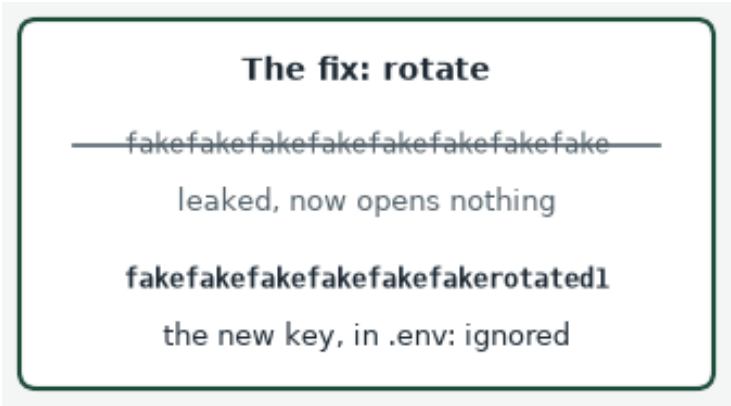
```
!git -C "{work}" show HEAD~1:settings.py
# a settings file with a key typed into it: the mistake the drill makes on purpose
DEMO_API_KEY = "fakefakefakefakefakefakefakefake"
```

```
%%writefile .env
# today's drill, after rotating: a new fake key replaces the leaked one
DEMO_API_KEY=fakefakefakefakefakefakerotated1
Overwriting .env
```

```
!git -C "{work}" status --short --ignored .env
ROTATED_FAKE_KEY = 'fakefakefakefakefakefakerotated1'
rotated = md.get_secret('DEMO_API_KEY', env_file='.env')
print('get_secret returns the new key:', rotated == ROTATED_FAKE_KEY)
!! .env
get_secret returns the new key: True
```

"!! .env": ignored, so rotating leaves nothing to commit; the leaked key still sits in "Add settings", opening nothing. Rewriting history to take a file out of every commit exists, but it is slow, changes every later commit, and never reaches copies already made: named here, not done.

Deleting a committed key leaves it in the history; rotating first, the old key removed, makes every copy worthless.



GitHub's Order

GitHub, which scans repositories for leaked keys, gives the same order: its documentation says to "rotate the affected credential immediately", and adds: "While you can also remove secrets from your Git history, this is time-intensive and often unnecessary if you've already revoked the credential."

Story sources: GitHub Docs, "About secret scanning", https://docs.github.com/en/code-security/secret-scanning/introduction/about-secret-scanning, and "Removing sensitive data from a repository", https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository (both read 2026-10-04).

Bring it home: your real key, in my_bondmath

Give your real key its home, and bring "marketdata.py" into "my_bondmath", replacing any there. The work folder's ".gitignore" and ".env.example" are the drill's: leave them. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
Set-Location "$HOME\projects\python-fixed-income"
python -m pip install -r course03_market_data\requirements.txt
Set-Location "$HOME\projects\my_bondmath"
Add-Content .gitignore ".env", "cache/", "*.db"
Set-Content .env.example "FRED_API_KEY="
code .env
```

In the editor type "FRED_API_KEY=", paste your key after the "=", and save: it never touches PowerShell's history.

```
git check-ignore -v .env
git status --short --ignored
$work = "$env:TEMP\pfi_course3_01_k4w7q2xr"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
python -m pytest -q test_marketdata.py
git add .gitignore .env.example marketdata.py test_marketdata.py
git commit -m "Notebook 01: get_secret, a .gitignore, and .env.example"
```

pytest shows "4 passed"; ".env" is never in the "git add" line. Restart this notebook and run all cells: the key cell says "found", and the live cell pulls FRED. In Colab, download the two files from the work folder under "/tmp"; the key stays in the Secrets panel.

You Can Now

Take a request apart: endpoint, parameters, status code, body.

Get your own FRED key, and keep it like a password.

Keep a secret in the environment or a ".env" file, and compare it instead of printing it.

Put ".gitignore" in place before the first commit, and prove it with "git check-ignore -v".

Find a secret in three places with "get_secret", tested with "monkeypatch" and "tmp_path".

Find a deleted key in Git's history, and rotate it.

Predict first: an unissued key, and two places

Predict 1

You send that request with a key of the right shape that FRED never issued. What status code comes back? Write down a number.

```
params = {'series_id': 'DGS10', 'api_key': FAKE_KEY, 'file_type': 'json',
          'observation_start': '2026-09-01'}
request_url = FRED_URL + '?' + '&'.join([f'{name}={value}' for name, value in params.items()])
print(request_url)
```

Predict 2

Set "DEMO_API_KEY" in the environment to another value, then read the same ".env" with "dotenv_values". The dictionary holds:

A) the environment's value

B) the file's value

C) an error: one name, two values

```
os.environ['DEMO_API_KEY'] = 'a-value-in-the-environment'
values_again = dotenv_values('.env')
```

Predict first: an ignored file, and the whole history

Predict 3

"settings.py" is committed and now listed in ".gitignore". Does "git status" report the new line?

A) No: Git looks away from an ignored name

B) Yes: Git still watches the file

C) Git refuses the edit

```
!git -C "{work}" status --short settings.py
```

Predict 4

Search again with no file named. How many commits does Git list? Write down a number.

```
!git -C "{work}" log --oneline -S {FAKE_KEY}
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: describe_request's test, written from its docstring, fails: AssertionError: the log line holds the API key.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
What FRED sends back	Text: 43 observations, 5.29 on 2026-09-30, the snapshot's value
Your own key	Yours alone, in my_bondmath\.env or Colab's Secrets panel
A home outside files and commits	.env ignored before the first commit
The function, tested	get_secret, 4 passed
The 10-year live	Tied to the snapshot on every shared date
A leak, and the fix	Still in the history after the delete; rotate first

Be proud: every key you handle from here gets these habits.

Next: JSON and the FRED API (notebook 02) turns the text from 1.1 into a dated series, and one live cell into three tested functions.